SOMAIYA
VIDYAVIHAR

K J Somaiya Institute of Technology
(Formerly known as K J Somaiya Institute of Engineering and Information Technology)
An Autonomous Institute Permanently Affiliated to University of Mumbai.

Somaiya TRUST

Department of Information Technology.

Cover page ASSIGNMENT Academic Year: 26-27 Class: SY IT Sem: III

Subject: Data Structurs & Analysis Course Code:

Name: Anushka Shettigar Roll No: BS2 - 36

ASSIGNMENT NO : 1/2 ✓	Date of Performance: 25/08/26	Date of Correction: 20/9/26

Rubrics	Marks (20)	Grade	Signature of Faculty
Participation and Involvement (5)			
Preparation and Knowledge (5)			
Accuracy and neatness of Documentation(5)			
Punctuality (5)			

Grades: Excellent – E (17 – 20)
Very Good – A (13-16)
Average – B (9-12)
Satisfactory – C (5-8)
Unsatisfactory – D (< - 5)

DSA SY IT B

Assignment 1

Q1 Climate Heatwave Monitoring Using Queue Design and implement a menu-driven C program using an array based Queue to manage weather observations for a Climate Heatwave Monitoring System. Each Observation should contain Observation ID, Location, Timestamp, Temperature, and Humidity, Implement Enqueue, Dequeue, Peek Display, Search, Count and Queue Status Operations. Demonstrate FIFO processing and analyze the time and space complexity of the Queue operations.

Ans Program code:

```
#include <stdio.h>
#include <string.h>

#define MAX 100

typedef struct {
    int id;
    char location[50];
    char timestamp[20];
    float temperature;
    float humidity;
} Observation;

typedef struct {
    Observation items[MAX];
    int front;
    int rear; } Queue;
```

```
void initQueue (Queue *q){
    q -> front = -1;
    q -> rear = -1;
}

int isFull (Queue*q){
    return q -> rear == MAX-1;

int isEmpty(Queue *q){
    return q -> front == -1 || q -> front > q -> rear;
}

void enqueue (Queue*q){
    if (isFull(q)){
    printf("\n Queue Overflow! Cannot add more
                observations.\n");
    return;
}
if (q -> front == -1){
        q -> front = 0;
}
q -> rear++;
Observation *obs = &q -> items [q -> rear];

printf ("\nEnter Observation ID: ");
scanf ("%d", &obs -> id);
printf("Enter Location: ");
scanf ("%[^\n]s", obs -> location);
printf ("Enter Timestamp (e.g.,2026-08-25 14:00): ");
scanf ("%[^\n]s", obs -> timestamp);
printf("Enter Temperature (°C): ");
```

Analysis of Time & Space Complexity.

Operation	Time Complexity	Space Complexity	~~Explanati~~
Enqueue	O(1)	O(1)	
Dequeue	O(1)	O(1)	
Peek	O(1)	O(1)	
Display	O(N)	O(1)	
Search	O(N)	O(1)	
Count	O(1)	O(1)	

DSA 1 : Sorting Performance Comparison Table

Algorithm	Dataset Size	Execution Time	No. of Comparisons
Bubble ~~Cat~~ Sort	100	0.08 ms	4950
Insertion Sort	100	0.04 ms	42475
Selection ~~Cat~~ Sort	100	0.05 ms	4950
Merge Sort	100	0.03 ms	540
Quick Sort	100	0.02 ms	480
Bubble Sort	500	1.85 m	124750
Selection Sort	500	1.10 ms	124,750
Insertion Sort	500	0.85 ms	62,375
Merge Sort	500	0.18 ms	3816
Quick Sort	500	0.11 ms	3210

```
int Queue(&q);
 int choice;
do {
    printf("\n=========================");
    printf("\n CLIMATE HEATWAVE MONITORING SYSTEM");
    printf("\n=========================");
    printf("\n 1. Enqueue (Add observation)");
    printf("\n 2. Dequeue (Process / Remove observation)
        ");
    printf("\n 3. Peek (View Oldest Observation)");
    printf("\n 4. Display (View All)");
    printf("\n 5. Search Observation");
    printf("\n 6. Count Observations");
    printf("\n 7. Queue Status");
    printf("\n 8. Exit");
    printf("\n Enter choice (1-8): ");
    scanf("%d", &choice);

    switch(choice){
    case 1: enqueue(&q); break;
    case 2: dequeue(&q); break;
    case 3: peek(&q); break;
    Case 4: display(&q); break;
    Case 5: search(&q); break;
    case 6: count(&q); break;
    case 7: queuestatus(&q); break;
    case 8; printf("\n Exiting program...\n"); break;
    default: printf("\n Invalid Choice! Try Again.\n");
    }} while (Choice != 8);
    return 0;
}
```

```
q -> items[i].id, q -> items[i].location, q -> items[i].
tempeature, q -> items[i].humidity);
found=1;
break;
 }
}
if (!found) {
   printf ("Observation ID %.d not found. \n", searchID);
   }
}
void count (Queue *q){
    if (isEmpty(q)) {
    printf ("\n Total Observations: 0 \n");
  } else {
    printf("\n Total Observations: %.d \n", (q->rear-
        q->front +1));
       }
}
void queueStatus (Queue *q){
     printf ("\n --- Queue Status ---");
  if (isEmpty (q)){
    printf ("\n Status: EMPTY \n");
  }else if (isFull(q)){
   printf ("\n Status: FULL \n");
  } else { printf ("\n Status : Partially Filled
        (%.d/%.d elements) \n", (q->rear - q->front +1),
        MAX);
    }
}
int main(){
    Queue q;
```


Department of: - /Class /Sem. /Sub. /Academic Year:-

```
obs.humidity);

Void display(Queue *q){
    If (isEmpty(q)) {
        printf("\n Queue is empty. \n");
        return;
    }.
printf("\n --- All Observations in Queue (FIFO order)
        ---\n"));
for(int i = q -> front; i <= q -> rear; i++){
 printf("ID: %d | Location: %-10s | Time: %-16s | Temp: %.
        1f °C | Humidity: %.1f%%\n",
        q -> items[i].id, q -> items[i].location, q -> items
        [i].timestamp, q->items[i].temperature, q -> items[i].
        humidity);

Void search (Queue *q){
    If (isEmpty(q)) {
    printf("\n Queue is empty.\n");
    return;
}
int searchID, found = 0;
printf("\n Enter Observation ID to search: ");
scanf("%d", &searchID);

for(int i = q -> front; i <= q -> rear; i++) {
    if (q -> items[i].id == searchID){
printf("\n Record Found at Position %d:", (i - q -> front
                                          +1));

printf("\nID: %d | location: %s | Temp: %.1f °C |
        Humidity: %.1f%%\n",
```

```
scanf("%f", &obs->temperature);
printf("Enter Humidity (%%): ");
scanf("%f", &obs->humidity);

printf("Observation ID %d added succesfully.\n",
        obs->id);
}

# void dequeue (Queue *q) {
  if (isEmpty(q)) {
    printf("\nQueue Underflow! No observations to
      remove.\n");
    return;
  }
  Observation obs = q->items[q->front];
  printf("\n Processed & Removed observation:");
  printf("\n ID: %d | Location: %s | Temp: %.1f°C |
      Humidity: %.1f%%\n", obs.id, obs.location,
    obs.temperature, obs.humidity);
    q->front++;
  }


void peek(Queue *q) {
    if (isEmpty(q)) {
      printf("\nQueue is empty.\n");
      return;
    }
  Observation obs = q->items[q->front];
  printf("\n -- Earliest (Front) Observation ---");
  printf("\n ID: %d \n Location: %s\n Timestamp: %s\n
      Temperature: %.1f°C\n Humidity: %.1f%%\n",
    obs.id, obs.location, obs.timestamp, obs.temperature
                                                    2 );
```